

Modularité en Python

Qu'est-ce que la Modularité ?

La modularité = **découper son programme en petits morceaux** (modules) indépendants.

Analogie

Imaginez une bibliothèque :

- ❑ Chaque **étagère** = un module
- ❑ Chaque **livre** = une fonction/variable
- ❑ Le **catalogue** = les imports

Pourquoi c'est utile ?

- ❑ Plus facile à **trouver** son code
- ❑ Plus simple à **corriger** les erreurs
- ❑ On peut **réutiliser** le même code dans plusieurs projets
- ❑ Travail à **plusieurs** sans se marcher sur les pieds

C'est Quoi un Module ?

Un module = un fichier Python

mon_premier_module.py ← C'est un module !

Exemple concret

```
# C'est mon premier module !

def dire_bonjour():
    print("Salut !")

def dire_bonsoir():
    print("Bonne soirée !")

auteur = "Moi-même"
```

Rien de compliqué ! Un simple fichier .py avec des fonctions dedans.

Comment Utiliser un Module ?

Pour utiliser un module, on l'importe !

Méthode 1 - Importer tout le module

```
import bienvenue

bienvenue.dire_bonjour() # Salut !
print(bienvenue.auteur)  # Moi-même
```

Méthode 2 - Importer seulement ce dont on a besoin

```
from bienvenue import dire_bonjour, auteur

dire_bonjour()      # Salut !
print(auteur)        # Moi-même
# dire_bonsoir()     # Erreur ! Non importée
```

Méthode 3 - Donner un surnom (alias)

```
import bienvenue as bv

bv.dire_bonjour()    # Salut !
```

À Quoi Sert if `__name__ == "__main__"` ?

Une variable magique : `__name__`

Dans `test.py` :

```
print(f"Je m'appelle {__name__}")
```

Quand on exécute directement :

```
python test.py  
# Affiche : Je m'appelle __main__
```

Quand on importe :

```
import test  
# Affiche : Je m'appelle test
```

L'astuce géniale

```
# mon_module.py  
def ma_fonction():  
    print("Je suis une fonction")  
  
# Ce code ne s'exécute que si on lance ce fichier directement  
if __name__ == "__main__":  
    print("Test de mon module")  
    ma_fonction()
```

Utilité : On peut tester son module sans que le test s'exécute quand on l'importe ailleurs !

Premier Projet Modulaire

Créons un mini projet : Calculatrice simple

```
calculatrice/  
├─ operations.py  
└─ main.py
```

```
# operations.py  
def addition(a, b):  
    return a + b  
  
def soustraction(a, b):  
    return a - b  
  
def multiplication(a, b):  
    return a * b  
  
def division(a, b):  
    if b == 0:  
        return "Erreur: division par zéro"  
    return a / b
```

```
# main.py  
import operations  
  
def afficher_menu():  
    print("\n--- CALCULATRICE ---")  
    print("1. Addition")  
    print("2. Soustraction")  
    print("3. Multiplication")  
    print("4. Division")  
    print("5. Quitter")  
  
def main():  
    while True:  
        afficher_menu()  
        choix = input("Choisissez une opération: ")  
  
        if choix == "5":  
            print("Au revoir !")  
            break  
  
        if choix in ["1", "2", "3", "4"]:  
            a = float(input("Premier nombre: "))  
            b = float(input("Deuxième nombre: "))  
  
            if choix == "1":  
                resultat = operations.addition(a, b)  
            elif choix == "2":  
                resultat = operations.soustraction(a, b)  
            elif choix == "3":  
                resultat = operations.multiplication(a, b)  
            else:  
                resultat = operations.division(a, b)  
  
            print(f"Résultat: {resultat}")  
        else:  
            print("Choix invalide")  
  
if __name__ == "__main__":  
    main()
```

Les Packages

Package = dossier qui contient des modules

À quoi ça sert ?

- regrouper plusieurs modules
- mieux organiser un projet
- créer une structure claire
- éviter de mettre tout dans un seul fichier
- faciliter la réutilisation du code

Structure :

```
mon_package/  
├── __init__.py    ← Fichier OBLIGATOIRE (peut être vide)  
├── module1.py  
├── module2.py  
└── sous_package/  ← Package dans un package  
    ├── __init__.py  
    └── module3.py
```

Le fichier `__init__.py` dit à Python : "Ce dossier est un package !"

```
import mon_package.module1  
from mon_package import module2  
from mon_package.sous_package import module3
```

Exemple de Package

```
utils/  
├── __init__.py  
├── texte.py  
└── maths.py
```

utils/texte.py :

```
def majuscules(texte):  
    return texte.upper()  
  
def minuscules(texte):  
    return texte.lower()  
  
def compter_mots(texte):  
    return len(texte.split())
```

utils/maths.py :

```
def carre(n):  
    return n ** 2  
  
def cube(n):  
    return n ** 3  
  
def est_pair(n):  
    return n % 2 == 0
```

Utiliser un Package

Méthode 1 : Importer un module du package

```
from utils import texte  
  
print(texte.majuscules("bonjour")) # BONJOUR  
print(texte.compter_mots("J'aime Python")) # 3
```

Méthode 2 : Importer directement une fonction

```
from utils.texte import majuscules, minuscules  
  
print(majuscules("python")) # PYTHON  
print(minuscules("PYTHON")) # python
```

Méthode 3 : Importer plusieurs modules

```
from utils import texte, maths  
  
print(texte.majuscules("hello")) # HELLO  
print(maths.carre(5)) # 25
```

Imports Relatifs

Dans un package, on peut importer entre modules

Dans module_a.py

```
mon_package/  
├── __init__.py  
├── module_a.py  
└── module_b.py
```

```
# module_a.py  
def fonction_a():  
    return "A"  
  
def fonction_commune():  
    return "Commune A"
```

Dans module_b.py :

```
# module_b.py  
from . import module_a      # . = même dossier  
from .module_a import fonction_a  
  
def fonction_b():  
    return "B"  
  
def fonction_commune():  
    return "Commune B"  
  
def utiliser_a():  
    return fonction_a() + " + " + fonction_b()
```

Les imports relatifs ne marchent que dans les packages !

Projet Complet - Gestionnaire de Tâches

```
todo/  
├─ main.py      # Programme principal  
├─ taches.py    # Gestion des tâches  
├─ fichiers.py  # Sauvegarde/chargement  
└─ affichage.py # Affichage des menus
```

```
# fichiers.py  
import json  
  
def sauvegarder_taches(taches, nom_fichier="taches.json"):  
    """Sauvegarde les tâches dans un fichier JSON"""  
    try:  
        with open(nom_fichier, 'w') as f:  
            json.dump(taches, f)  
        return True  
    except:  
        return False  
  
def charger_taches(nom_fichier="taches.json"):  
    """Charge les tâches depuis un fichier JSON"""  
    try:  
        with open(nom_fichier, 'r') as f:  
            return json.load(f)  
    except:  
        return [] # Retourne liste vide si erreur
```

```
# taches.py  
taches = [] # Liste globale des tâches  
  
def ajouter_tache(titre):  
    """Ajoute une nouvelle tâche"""  
    taches.append({"titre": titre, "faite": False})  
    return True  
  
def supprimer_tache(index):  
    """Supprime une tâche par son index"""  
    if 0 <= index < len(taches):  
        del taches[index]  
        return True  
    return False  
  
def marquer_faite(index):  
    """Marque une tâche comme terminée"""  
    if 0 <= index < len(taches):  
        taches[index]["faite"] = True  
        return True  
    return False  
  
def obtenir_taches():  
    """Retourne la liste des tâches"""  
    return taches
```

Projet Complet - Gestionnaire de Tâches

```
# affichage.py
def afficher_menu():
    """Affiche le menu principal"""
    print("\n" + "="*40)
    print("    GESTIONNAIRE DE TÂCHES")
    print("="*40)
    print("1. Ajouter une tâche")
    print("2. Voir les tâches")
    print("3. Marquer une tâche comme faite")
    print("4. Supprimer une tâche")
    print("5. Sauvegarder")
    print("6. Charger")
    print("7. Quitter")
    print("="*40)

def afficher_taches(taches):
    """Affiche toutes les tâches"""
    if not taches:
        print("\nAucune tâche pour le moment")
        return

    print("\nVOS TACHES :")
    for i, tache in enumerate(taches):
        statut = "[X]" if tache["faite"] else "[ ]"
        print(f"{i+1}. {statut} {tache['titre']}")
```

Projet Complet - Gestionnaire de Tâches

```
from taches import *
from fichiers import sauvegarder_taches, charger_taches
from affichage import afficher_menu, afficher_taches

def main():
    """Programme principal"""
    # On charge les tâches existantes
    global taches
    taches = charger_taches()

    while True:
        afficher_menu()
        choix = input("\nVotre choix : ")

        if choix == "1":
            titre = input("Titre de la tâche : ")
            ajouter_tache(titre)
            print(f"[OK] Tâche '{titre}' ajoutée !")

        elif choix == "2":
            afficher_taches(taches)

        elif choix == "3":
            afficher_taches(taches)
            try:
                index = int(input("Numéro de la tâche à terminer : "))
                if marquer_faites(index):
                    print("[OK] Tâche marquée comme faite !")
                else:
                    print("[ERREUR] Index invalide")
            except ValueError:
                print("[ERREUR] Veuillez entrer un nombre")

        elif choix == "4":
            afficher_taches(taches)
            try:
                index = int(input("Numéro de la tâche à supprimer : ")) - 1
                if supprimer_tache(index):
                    print("[SUPPRIMER] Tâche supprimée !")
                else:
                    print("[ERREUR] Index invalide")
            except ValueError:
                print("[ERREUR] Veuillez entrer un nombre")

        elif choix == "5":
            if sauvegarder_taches(taches):
                print("[SAUVEGARDE] Sauvegarde réussie !")
            else:
                print("[ERREUR] Erreur lors de la sauvegarde")

        elif choix == "6":
            taches = charger_taches()
            print("[CHARGEMENT] Chargement réussi !")

        elif choix == "7":
            # Sauvegarde automatique avant de quitter
            sauvegarder_taches(taches)
            print("[AU REVOIR] Au revoir !")
            break

        else:
            print("[ERREUR] Choix invalide")

if __name__ == "__main__":
```

Modules Python Intégrés

```
import math

print(math.sqrt(16))      # 4.0 (racine carrée)
print(math.pi)           # 3.14159...
print(math.floor(3.7))    # 3 (arrondi inférieur)
print(math.ceil(3.2))     # 4 (arrondi supérieur)
```

```
import random

print(random.randint(1, 10)) # Nombre entre 1 et 10
print(random.choice(['a', 'b', 'c'])) # Choix aléatoire
```

```
import os

print(os.getcwd())        # Dossier actuel
print(os.listdir('.'))    # Liste des fichiers
os.mkdir('nouveau_dossier') # Créer un dossier
```

```
import datetime

print(datetime.datetime.now()) # Date et heure actuelles
print(datetime.date.today())  # Date du jour
```

Module **os** - Interface avec le système d'exploitation

```
import os

# Lister le contenu du dossier actuel
contenu = os.listdir(".")
print("Contenu du dossier :")
for element in contenu:
    print(f" - {element}")

# Lister un dossier spécifique
fichiers = os.listdir("/home/utilisateur/Documents")
```

```
import os

# Créer un dossier
os.mkdir("nouveau_dossier")
print("Dossier créé")

# Créer une arborescence complète
os.makedirs("dossier1/sous_dossier2/sous_sous_dossier3")

# Supprimer un dossier VIDE
os.rmdir("nouveau_dossier")

# Supprimer une arborescence (dossiers vides)
os.removedirs("dossier1/sous_dossier2/sous_sous_dossier3")
```

```
import os

# Obtenir le répertoire de travail actuel
chemin_actuel = os.getcwd()
print(f"Je suis dans : {chemin_actuel}")

# Changer de répertoire
os.chdir("/chemin/vers/dossier")
print(f"Nouveau répertoire : {os.getcwd()}")

# Revenir au dossier parent
os.chdir("..")
```

```
import os

# Supprimer un fichier
if os.path.exists("fichier.txt"):
    os.remove("fichier.txt")
    print("Fichier supprimé")
```

Module os - Commandes système

```
import os

# Exécuter une commande système
os.system("ls -l") # Linux/Mac
os.system("dir")   # Windows

# Alternative plus moderne (recommandée)
import subprocess

# Capturer la sortie
resultat = subprocess.run(["ls", "-l"], capture_output=True, text=True)
print(resultat.stdout)

# Avec gestion d'erreur
try:
    subprocess.run(["commande_inexistante"], check=True)
except subprocess.CalledProcessError as e:
    print(f"Erreur : {e}")
```

```
import sys
```

```
# sys.argv contient les arguments passés au script
```

```
print(f"Nom du script : {sys.argv[0]}")
```

```
print(f"Arguments : {sys.argv[1:]}")
```

```
# Exemple d'utilisation
```

```
if len(sys.argv) > 1:
```

```
    print(f"Bonjour {sys.argv[1]}!")
```

```
else:
```

```
    print("Donnez un nom en argument")
```

```
python mon_script.py Alice
```

```
# Affiche : Bonjour Alice!
```

```
import sys
```

```
# Sortie standard
```

```
sys.stdout.write("Message normal\n")
```

```
# Sortie d'erreur
```

```
sys.stderr.write("Message d'erreur\n")
```

```
# Quitter le programme
```

```
sys.exit(0)      # Succès
```

```
sys.exit(1)      # Erreur
```

Installer des Modules Externes

pip = le gestionnaire de paquets Python

Exemple avec requests

```
# Installer un module
pip install requests

# Installer une version spécifique
pip install requests==2.28.1

# Désinstaller
pip uninstall requests

# Voir tous les modules installés
pip list
```

```
import requests

# Récupérer une page web
reponse = requests.get("https://api.github.com")

print(reponse.status_code) # 200 = OK
print(reponse.json())      # Contenu en JSON
```

Traiter la réponse

```
response = requests.get('https://api.example.com/users')

# Vérifier le statut
print(response.status_code) # 200, 404, 500...
print(response.ok)         # True si 200-400

# Récupérer les données
if response.ok:
    data = response.json() # Si réponse JSON
    text = response.text   # Texte brut
    content = response.content # Binaire (image, etc.)
```

GET (récupérer des données)

```
# Simple
response = requests.get('https://api.example.com/users')

# Avec paramètres
params = {'page': 2, 'limit': 10}
response = requests.get('https://api.example.com/users', params=params)

# Avec headers
headers = {'Authorization': 'Bearer token123'}
response = requests.get('https://api.example.com/users', headers=headers)
```

POST (créer des données)

```
# JSON
data = {'name': 'Jean', 'email': 'jean@email.com'}
response = requests.post('https://api.example.com/users', json=data)

# Formulaire
form_data = {'username': 'jean123', 'password': 'secret'}
response = requests.post('https://api.example.com/login', data=form_data)
```

PUT (mettre à jour)

```
data = {'name': 'Jean Dupont'}
response = requests.put('https://api.example.com/users/1', json=data)
```

DELETE (supprimer)

```
response = requests.delete('https://api.example.com/users/1')
```

Gestion des erreurs

```
try:
    response = requests.get('https://api.example.com/data', timeout=5)
    response.raise_for_status() # Lève une exception si erreur (4xx, 5xx)
    data = response.json()
except requests.exceptions.RequestException as e:
    print(f"Erreur: {e}")
```

Exemple complet

```
import requests

API_URL = "https://jsonplaceholder.typicode.com/posts"

# GET
response = requests.get(f"{API_URL}/1")
post = response.json()
print(post['title'])

# POST
new_post = {
    'title': 'Mon article',
    'body': 'Contenu...',
    'userId': 1
}
response = requests.post(API_URL, json=new_post)
print(f"Créé: {response.status_code}")

# Vérification
if response.status_code == 201:
    print("Succès!")
```

Méthodes utiles

Méthode	Description
<code>response.status_code</code>	Code HTTP
<code>response.json()</code>	Parse JSON
<code>response.text</code>	Texte brut
<code>response.headers</code>	En-têtes HTTP
<code>response.raise_for_status()</code>	Lève exception si erreur

Tableau des exceptions principales

Exception	Quand elle se produit
<code>ConnectionError</code>	DNS résolu mais connexion impossible (serveur down, refus de connexion)
<code>Timeout</code>	Requête trop longue (délai dépassé)
<code>HTTPError</code>	Code HTTP 4xx ou 5xx (quand <code>raise_for_status()</code> est appelé)
<code>TooManyRedirects</code>	Trop de redirections
<code>RequestException</code>	Classe parente de toutes les exceptions requests
<code>JSONDecodeError</code>	<code>response.json()</code> appelé sur une réponse non JSON

Python sur Tkinter

Tkinter sert à créer des applications graphiques. C'est le module GUI standard de Python, disponible sur Windows, macOS et la plupart des systèmes Unix. **Widget** = élément graphique qu'on place dans une fenêtre.

Les widgets de base en Tkinter, c'est surtout :

- **Label** : affiche du texte
- **Button** : bouton cliquable
- **Entry** : champ pour écrire une ligne
- **Text** : zone de texte plus grande, sur plusieurs lignes
- **Frame** : conteneur pour regrouper des widgets
- **Checkbutton** : case à cocher
- **Radiobutton** : choix unique parmi plusieurs options
- **Listbox** : liste d'éléments
- **Combobox** : liste déroulante
- **Canvas** : zone pour dessiner
- **Menu** : barre de menu
- **Scrollbar** : barre de défilement

```
import tkinter as tk

# Créer la fenêtre principale
root = tk.Tk()
root.title("Ma première application")
root.geometry("400x300") # Largeur x hauteur

# Ajouter un Label (texte)
label = tk.Label(root, text="Bonjour Tkinter !")
label.pack()

# Ajouter un bouton
def on_click():
    print("Bouton cliqué !")

button = tk.Button(root, text="Cliquez-moi", command=on_click)
button.pack()

# Lancer l'application
root.mainloop()
```

tk.Tk() : crée la fenêtre principale

.mainloop() : boucle d'événements qui maintient la fenêtre ouverte

```

import tkinter as tk

root = tk.Tk()

# Label
label = tk.Label(root, text="Nom :")
label.grid(row=0, column=0, padx=5, pady=5)

# Entry
entry = tk.Entry(root)
entry.grid(row=0, column=1, padx=5, pady=5)

# Text (multiligne)
text = tk.Text(root, height=5, width=30)
text.grid(row=1, column=0, columnspan=2, padx=5, pady=5)

# Button
def get_text():
    print(f"Nom: {entry.get()}")
    print(f"Message: {text.get('1.0', tk.END)}")

button = tk.Button(root, text="Valider", command=get_text)
button.grid(row=2, column=0, columnspan=2, pady=10)

root.mainloop()

```

Ce code crée une petite interface graphique avec une fenêtre contenant un champ de saisie, une zone de texte multiligne et un bouton. Voici comment il fonctionne

Au-delà de Tkinter : wxPython et PyQt

Pourquoi passer à autre chose ?

Tkinter est simple et inclus dans Python, mais son rendu paraît souvent plus ancien et son catalogue de widgets reste plus limité pour des interfaces riches.

wxPython

Installation	<code>pip install wxPython</code>
Apparence	Native : suit le style Windows, macOS ou Linux
Licence	wxWindows : libre et souple pour les projets

+ Points forts

- Très bonne intégration visuelle au système
- Riche en widgets prêts à l'emploi

– Limites

- Moins de ressources et d'exemples que Qt

PyQt

Installation	<code>pip install PyQt5</code>
Apparence	Native ou fortement personnalisable via styles
Licence	GPL pour l'open source ou licence commerciale

+ Points forts

- Framework Qt très vaste et mature
- Qt Designer pour créer l'interface en glisser-déposer
- Système signaux/slots élégant

– Limites

- Licence plus contraignante pour du code fermé

Cours HTML/CSS pour débutant

Objectif : comprendre la structure d'une page web, savoir la styliser, puis préparer le terrain pour le web scraping en Python.

```
<!DOCTYPE html>    ← Je dis au navigateur : "c'est du HTML5"
<html>              ← Je commence la page
<head>              ← En-tête (infos pour le navigateur)
  <title>Mon site</title> ← Titre dans l'onglet
</head>
<body>              ← Corps (ce que l'utilisateur VOIT)
  <h1>Bienvenue</h1>
  <p>Du texte</p>
</body>
</html>             ← Je ferme la page
```

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ma page</title>
  </head>
  <body>
    <h1>Bonjour le monde !</h1>
    <p>Ceci est ma première page web.</p>
  </body>
</html>
```

HTML

CSS

Python

HTML = structure · CSS = apparence · Python
= extraction plus tard

```
<!DOCTYPE html>
<html>
<head>
  <title>Ma page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f0f0;
      padding: 20px;
    }
    h1 {
      color: blue;
    }
    p {
      color: green;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <h1>Bonjour le monde !</h1>
  <p>Ceci est ma première page web.</p>
</body>
</html>
```

HTML, CSS, JavaScript : quelle différence ?

HTML

La structure

- Titres
- Paragraphes
- Liens
- Images
- Formulaires

CSS

L'apparence

- Couleurs
- Tailles
- Marges
- Disposition
- Responsive

JavaScript

Le comportement

- Cliquer
- Ouvrir un menu
- Valider un formulaire
- Mettre à jour la page

Pour le scraping, HTML est prioritaire. CSS vient juste après, car les sélecteurs servent souvent à cibler les éléments.

Les balises HTML les plus utiles au début

Balise	À quoi elle sert
<code><h1> à <h6></code>	Créer des titres hiérarchisés
<code><p></code>	Écrire un paragraphe
<code><a></code>	Créer un lien
<code></code>	Afficher une image
<code> / / </code>	Faire des listes
<code><div> / <section></code>	Structurer et regrouper du contenu

Astuce : lis toujours une balise comme une étiquette qui indique le rôle du contenu.

Les liens

<code><h1>Titre principal</h1></code>	← Très gros, un seul par page
<code><h2>Sous-titre</h2></code>	← Gros
<code><h3>Titre de section</h3></code>	← Moyen
<code><h4>Sous-section</h4></code>	← Petit
<code><h5>Détail</h5></code>	← Très petit
<code><h6>Note</h6></code>	← Tout petit

```
<!-- Lien vers un autre site -->
<a href="https://www.amazon.fr">Aller sur Amazon</a>

<!-- Lien vers une page du même site -->
<a href="/contact.html">Nous contacter</a>

<!-- Lien qui s'ouvre dans un nouvel onglet -->
<a href="https://google.com" target="_blank">Google</a>
```

Les images

```
<!-- Image simple -->


<!-- Image complète (recommandé) -->


<!-- Image avec info-bulle -->

```

```
<ul>
  <li>Pommes</li>
  <li>Poires</li>
  <li>Bananes</li>
</ul>
```

```
<ol>
  <li>Premier</li>
  <li>Deuxième</li>
  <li>Troisième</li>
</ol>
```

```
<div class="produit">
  <h2>iPhone 15</h2>
  <p>Le nouveau smartphone Apple</p>
  <span class="prix">999€</span>
</div>
```

Les tableaux

```
<table>
  <tr>
    <th>Nom</th>
    <th>Âge</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>25</td>
  </tr>
  <tr>
    <td>Bob</td>
    <td>30</td>
  </tr>
</table>
```

Les listes et les tables

```
<ul>
  <li>Pommes</li>
  <li>Poires</li>
  <li>Bananes</li>
</ul>
```

- Pommes
- Poires
- Bananes

```
<ol>
  <li>Premier</li>
  <li>Deuxième</li>
  <li>Troisième</li>
</ol>
```

1. Premier
2. Deuxième
3. Troisième

Nom	Âge
Alice	25
Bob	30

INTRODUCTION AU CSS

C'est quoi le CSS ?

Ce qui rend la page belle

Syntaxe CSS

```
selecteur {  
  propriete: valeur;  
  propriete: valeur;  
}
```

Exemple

```
h1 {  
  color: red;  
  font-size: 30px;  
}
```

Les 3 façons d'écrire du CSS

Méthode 1 : Dans la balise (inline)

```
<p style="color: red;">Texte rouge</p>
```

Méthode 3 : Fichier séparé (best practice)

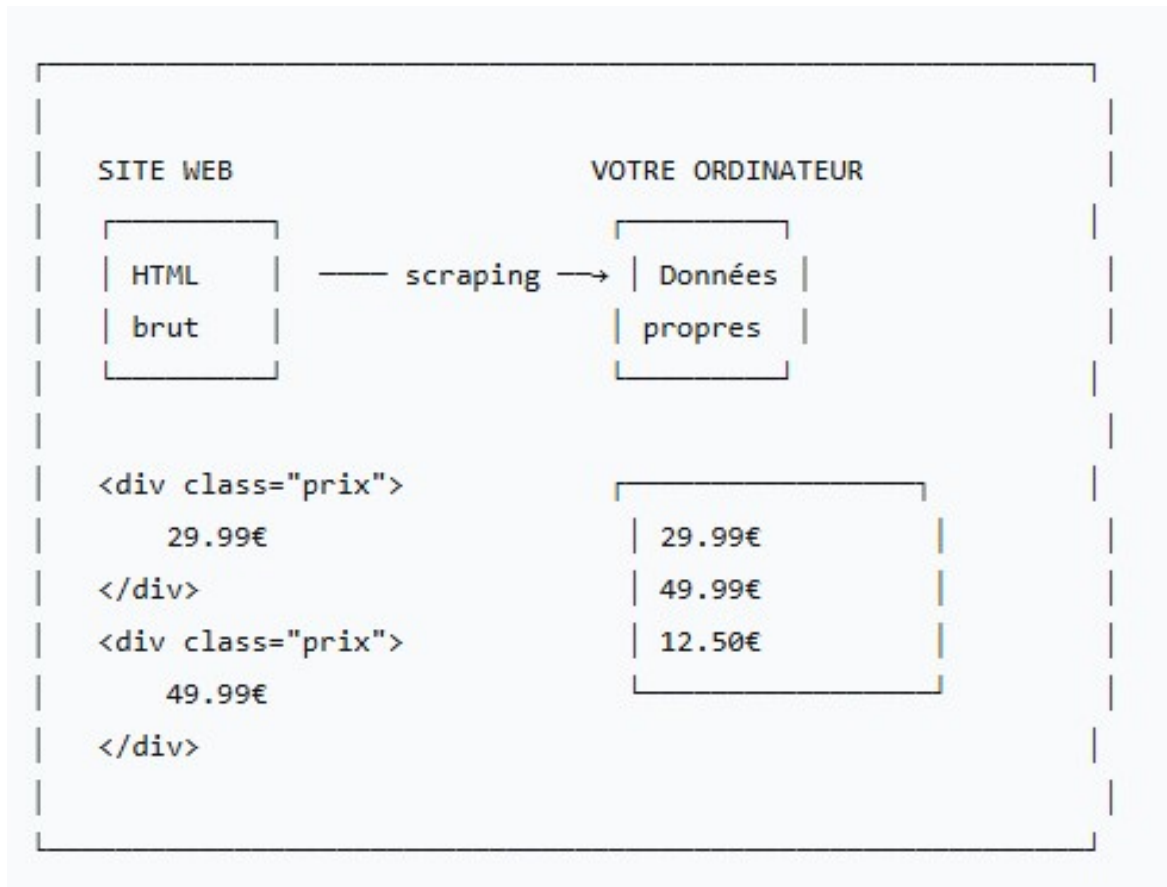
Méthode 2 : Dans la page (style)

```
<head>  
  <style>  
    p { color: red; }  
    .important { font-weight: bold; }  
  </style>  
</head>
```

```
<head>  
  <link rel="stylesheet" href="style.css">  
</head>
```

QU'EST-CE QUE LE **WEB SCRAPING** ?

Web scraping = extraire automatiquement des données d'un site web



PYTHON + 2 BIBLIOTHÈQUES

- ☐ BEAUTIFULSOUP
- ☐ REQUESTS

```
import requests
from bs4 import BeautifulSoup
```

Télécharger une page web

```
import requests

# URL de la page à scraper
url = "https://example.com"

# Envoyer une requête GET
reponse = requests.get(url)

# Vérifier que tout va bien
if reponse.status_code == 200:
    print("Page téléchargée !")
    html = reponse.text
    print(html[:500]) # Affiche les 500 premiers caracteres
else:
    print(f"Erreur : {reponse.status_code}")
```

Codes HTTP importants :

Code	Signification
200	OK (tout va bien)
404	Page non trouvée
403	Accès interdit
500	Erreur serveur

